CHAPTER 21
Advanced *sed*

IN THIS CHAPTER

Using multiline commands

Understanding the hold space

Negating a command

Changing the flow

Replacing via a pattern

Using sed in scripts

Making sed utilities

Chapter 19, "Introducing `sed` and `gawk`," showed you how to use the basics of the `sed` editor to manipulate text in data streams. The basic `sed` editor commands are capable of handling most of your everyday text-editing requirements. This chapter looks at the more advanced features that the `sed` editor has to offer. These are features that you might not use as often. But when you need them, it's nice to know that they're available as well as how to use them.

Looking at Multiline Commands

When using the basic `sed` editor commands, you may have noticed a limitation. All the `sed` editor commands perform functions on a single line of data. As the `sed` editor reads a data stream, it divides the data into lines based on the presence of newline characters. The `sed` editor handles the data lines one at a time, processing the defined script commands on the data line and then moving on to the next line and repeating the processing.

Sometimes, you need to perform actions on data that spans more than one line. This is especially true if you're trying to find or replace a phrase.

For example, if you're looking for the phrase `Linux System Administrators Group` in your data, it's quite possible that the phrase's words can be split onto two lines. If you processed the text using a normal `sed` editor command, it would be impossible to detect the split phrase.

Fortunately, the designers behind the `sed` editor thought of that situation and devised a solution. The `sed` editor includes three special commands that you can use to process multiline text:

- `N` adds the next line in the data stream to create a multiline group for processing.
- `D` deletes a single line in a multiline group.
- `P` prints a single line in a multiline group.

The following sections examine these multiline commands more closely and demonstrate how you can use them in your scripts.

Navigating the next command

Before you can examine the multiline next (`N`) command, you first need to look at how the single-line version of the next command works. After you know what that command does, it's much easier to understand how the multiline version of the next command operates.

Using the single-line next command

The single-line next (`n`) command tells the `sed` editor to move to the next line of text in the data stream, without going back to the beginning of the commands. Remember that normally the `sed` editor processes all the defined commands on a line before moving to the next line of text in the data stream. The single-line next (`n`) command alters this flow.

This may sound somewhat complicated, and sometimes it is. In this example, we have a data file that contains five lines, two of which are blank. The goal is to remove the first blank line, which is after the header line, but leave the second blank line intact. If we write a `sed` script to delete blank lines, *both* blank lines are removed, which is not what we wanted:

```
$ cat data1.txt
Header Line

Data Line #1

End of Data Lines
$
$ sed '/^$/d' data1.txt
Header Line
```

```
Data Line #1
End of Data Lines
$
```

Because the line we want to remove is blank, there is no text we can search for to uniquely identify the line. The solution is to use the single-line next (`n`) command. In this next example, the script looks for a unique line that contains the word `Header` . After the script identifies that line, the `n` command moves the `sed` editor to the next line of text, which is the blank line.

```
$ sed '/Header/{n ; d}' data1.txt
Header Line
Data Line #1

End of Data Lines
$
```

At that point, the `sed` editor continues processing the command list, which uses the `d` command to delete the empty line. When the `sed` editor reaches the end of the command script, it reads the next line of text from the data stream and starts processing commands from the top of the command script. The `sed` editor does not find another line with the word `Header` ; thus, no further lines are deleted.

Combining lines of text

Now that you've seen the single-line next (`n`) command, we can look at the multiline version. The single-line next command moves the next line of text from the data stream into the processing space (called the *pattern space*) of the `sed` editor. The multiline version of the next command (which uses a capital `N`) adds the next line of text to the text already in the pattern space.

This has the effect of combining two lines of text from the data stream into the same pattern space. The lines of text are still separated by a newline character, but the `sed` editor can now treat both lines of text as *one* line.

Here's a demonstration of how the multiline (`N`) command operates:

```
$ cat data2.txt
Header Line
First Data Line
Second Data Line
End of Data Lines
$
$ sed '/First/{ N ; s/\n/ / }' data2.txt
Header Line
First Data Line Second Data Line
End of Data Lines
$
```

The `sed` editor script searches for the line of text that contains the word `First` in it. When it finds the line, it uses the `N` command to combine the next line with that line in the pattern space. It then uses the substitution (`s`) command to replace the newline character (`\n`) with a space. The result is that the two lines in the text file appear as one line in the `sed` editor output.

This has a practical application if you're searching for a text phrase that may be split between two lines in the data file. Here's an example:

```
$ cat data3.txt
On Tuesday, the Linux System
Admin group meeting will be held.
All System Admins should attend.
Thank you for your cooperation.
$
$ sed 's/System Admin/DevOps Engineer/' data3.txt
On Tuesday, the Linux System
Admin group meeting will be held.
All DevOps Engineers should attend.
Thank you for your cooperation.
$
```

The substitution (`s`) command is looking for the specific two-word phrase `System Admin` in the text file. In the single line where the phrase appears, everything is fine; the substitution command can replace the text. But in the situation where the phrase is split between two lines, the substitution command doesn't recognize the matching pattern.

The `N` command helps solve this problem:

```
$ sed 'N ; s/System.Admin/DevOps Engineer/' data3.txt
On Tuesday, the Linux DevOps Engineer group meeting will be held.
All DevOps Engineers should attend.
Thank you for your cooperation.
$
```

By using the multiline (`N`) command to combine the next line with the line where the first word is found, you can detect when a line split occurs in the phrase.

Notice that the substitution (`s`) command uses a wildcard pattern (.) between the word `System` and the word `Admin` to match both the space and the newline situation. However, when it matched the newline character, it removed it from the string, causing the two lines to merge into one line. This may not be exactly what you want.

To solve this problem, you can use two substitution commands in the `sed` editor script, one to match the multiline occurrence and one to match the single-line occurrence:

```
$ sed 'N
> s/System\nAdmin/DevOps\nEngineer/
> s/System Admin/DevOps Engineer/
> ' data3.txt
On Tuesday, the Linux DevOps
Engineer group meeting will be held.
All DevOps Engineers should attend.
Thank you for your cooperation.
$
```

The first substitution command specifically looks for the newline character between the two search words and includes it in the replacement string. This allows you to add the newline character in the same place in the new text.

There's still one subtle problem with this script, however. The script always reads the next line of text into the pattern space before executing the `sed` editor commands. When it reaches the last line of text, there isn't a next line of text to read, so the `N` command causes the `sed` editor to stop. If the matching text is on the last line in the data stream, the commands don't catch the matching data:

```
$ cat data4.txt
On Tuesday, the Linux System
Admin group meeting will be held.
All System Admins should attend.
$
$ sed 'N
> s/System\nAdmin/DevOps\nEngineer/
> s/System Admin/DevOps Engineer/
> ' data4.txt
On Tuesday, the Linux DevOps
Engineer group meeting will be held.
All System Admins should attend.
$
```

Because the `System Admin` text appears in the last line in the data stream, the multiline (`N`) command misses it, as there isn't another line to read into the pattern space to combine. We can easily resolve this problem by moving our single-line editing commands before the multiline command and having only the editing commands for the multiple lines appear after the `N` , like this:

```
$ sed '
> s/System Admin/DevOps Engineer/
> N
> s/System\nAdmin/DevOps\nEngineer/
> ' data4.txt
On Tuesday, the Linux DevOps
Engineer group meeting will be held.
All DevOps Engineers should attend.
$
```

Now, the substitution (`s`) command that looks for the phrase in a single line works just fine on the last line in the data stream, and the substitution command after the multiline (`N`) command covers the occurrence in the middle of the data stream.

Navigating the multiline delete command

In Chapter 19, “Introducing sed and gawk,” we touched on the topic of the single-line delete (`d`) command. The `sed` editor uses it to delete the current line in the pattern space. If you're working with the `N` command, however, you must be careful when using the single-line delete command:

```
$ sed 'N ; /System\nAdmin/d' data4.txt
All System Admins should attend.
$
```

The delete (`d`) command looked for the words `System` and `Admin` in separate lines and deleted both of the lines in the pattern space. This may or may not have been what you intended.

The `sed` editor provides the multiline delete (`D`) command, which deletes only the first line in the pattern space. It removes all characters up to and including the newline character:

```
$ sed 'N ; /System\nAdmin/D' data4.txt
Admin group meeting will be held.
All System Admins should attend.
$
```

The second line of text, added to the pattern space by the `N` command, remains intact. This comes in handy if you need to remove a line of text that appears before a line that you find a data string in.

Here's an example of removing a blank line that appears before the first line in a data stream:

```
$ cat data5.txt

Header Line
First Data Line
```

```
End of Data Lines
$
$ sed '/^$/{N ; /Header/D}' data5.txt
Header Line
First Data Line

End of Data Lines
$
```

This `sed` editor script looks for blank lines and then uses the `N` command to add the next line of text into the pattern space. If the new pattern space contents contain the word `Header` , the `D` command removes the first line in the pattern space. Without the combination of the `N` and `D` commands, it would be impossible to remove the first blank line without removing all other blank lines.

Navigating the multiline print command

By now, you're probably catching on to the difference between the single-line and multiline versions of the commands. The multiline print command (`P`) follows along using the same technique. It prints only the first line in a multiline pattern space. This includes all characters up to the newline character in the pattern space. It is used in much the same way as the single-line `p` command to display text when you use the `-n` option to suppress output from the script.

```
$ sed -n 'N ; /System\nAdmin/P' data3.txt
On Tuesday, the Linux System
$
```

When the multiline match occurs, the `P` command prints only the first line in the pattern space. The power of the multiline `P` command comes into play when you combine it with the `N` and `D` multiline commands.

The `D` command has a unique feature in that after deleting the first line from the pattern space, it forces the `sed` editor to return to the beginning of the script and repeat the commands on the current pattern space (it doesn't read a new line of text from the data stream). By including the `N` command in the command script, you can effectively single-step through the pattern space, matching multiple lines together.

Next, by using the `P` command, you can print the first line, and then using the `D` command, you can delete the first line and loop back to the beginning of the script. When you are back at the script's beginning, the `N` command reads in the next line of text and starts the process all over again. This loop continues until you reach the end of the data stream, as shown in removing the data corruption in this file:

```
$ cat corruptData.txt
Header Line#
@
Data Line #1
Data Line #2#
@
End of Data Lines#
@
$
$ sed -n '
> N
> s/#\n@//
> P
> D
> ' corruptData.txt
Header Line
Data Line #1
Data Line #2
End of Data Lines
$
```

The data file has been corrupted with `#` at some lines' ends followed by `@` on the next line. To fix this issue, using `sed` the `Header Line#` line is loaded into the pattern space, and then the multiline next (`N`) command loads the second line (`@`), appending it to the first line within the space. The substitution (`s`) command removes the offending data (`#\n@`) by replacing it with a null. Next, the `P` command prints only the now cleaned-up first line within the pattern space. The delete (`D`) command removes this first line from the space and goes back to the beginning of the script, where the next `N` command reads the third line (`Data Line #1`) of text into the pattern space and the editing loop continues.

Holding Space

The *pattern space* is an active buffer area that holds the text examined by the `sed` editor while it processes commands. However, it isn't the only space available in the `sed` editor for storing text.

The `sed` editor utilizes another buffer area called the *hold space*. You can use the hold space to temporarily hold lines of text while working on other lines in the pattern space. The five commands associated with operating with the hold space are shown in Table 21.1.

TABLE 21.1 The sed Editor Hold Space Commands

Command	Description
h	Copies pattern space to hold space
H	Appends pattern space to hold space
g	Copies hold space to pattern space
G	Appends hold space to pattern space
x	Exchanges contents of pattern and hold spaces

These commands let you copy text from the pattern space to the hold space. This frees up the pattern space to load another string for processing.

Usually, after using the h or H command to move a string to the hold space, eventually you want to use the g , G , or x command to move the stored string back into the pattern space (otherwise, you wouldn't have cared about saving them in the first place).

With two buffer areas, trying to determine what line of text is in which buffer area can sometimes get confusing. Here's a short example that demonstrates how to use the h and g commands to move data back and forth between the sed editor buffer spaces:

```
$ cat data2.txt
Header Line
First Data Line
Second Data Line
End of Data Lines
$
$ sed -n '/First/ {
> h ; p ;
> n ; p ;
> g ; p }
> ' data2.txt
First Data Line
Second Data Line
First Data Line
$
```

Let's look at the preceding code example step by step:

1. The sed script uses a regular expression in the address to filter the line containing the word First .
2. When the line containing the word First appears, the initial command in {} , the h command, copies the line in the pattern space to the hold space. At this point, the pattern space and the hold space have the same data.
3. The p command then prints the contents of the pattern space (First Data Line), which is still the line that was copied into the hold space.
4. The n command retrieves the next line in the data stream (Second Data Line) and places it in the pattern space. Now the pattern space has *different* data than the hold space.
5. The p command prints the contents of the pattern space (Second Data Line).
6. The g command places the contents of the hold space (First Data Line) back into the pattern space, replacing the current text. The pattern space and the hold space now have the same data again.
7. The p command prints the current contents of the pattern space (First Data Line).

By shuffling the text lines around using the hold space, we can force the First Data Line to appear after the Second Data Line in the output. If we just drop the first p command, we can output the two lines in reverse order:

```
$ sed -n '/First/ {
> h ;
> n ; p
> g ; p }
> ' data2.txt
Second Data Line
First Data Line
$
```

This is the start of something useful. You can use this technique to create a sed script that reverses an entire file of text data! To do that, however, you need to see the negating feature of the sed editor, which is what the next section is all about.

Negating a Command

Chapter 19 showed that the sed editor applies commands either to every text line in the data stream or to lines specifically indicated by either a single address or an address range. You can also configure a command to *not* apply to a specific address or address range in the data stream.

The exclamation mark (!) command is used to negate a command. This means in situations where the command would normally have been activated, it isn't. Here's an example demonstrating this feature:

```
$ sed -n '/Header/!p' data2.txt
First Data Line
Second Data Line
End of Data Lines
$
```

The normal `p` command would have printed only the line in the `data2` file that contained the word `Header` . By adding the exclamation mark, the opposite happens — all lines in the file are printed except the one that contained the word `Header`.

Using the exclamation mark comes in handy in several applications. Recall that earlier in the chapter, the "Navigating the Next Command" section showed a situation where a `sed` editor command wouldn't operate on the last line of text in the data stream because there wasn't a line after it. You can use the exclamation point to fix that problem:

```
$ cat data4.txt
On Tuesday, the Linux System
Admin group meeting will be held.
All System Admins should attend.
$
$ sed 'N;
> s/System\nAdmin/DevOps\nEngineer/
> s/System Admin/DevOps Engineer/
> ' data4.txt
On Tuesday, the Linux DevOps
Engineer group meeting will be held.
All System Admins should attend.
$
$ sed '$!N;
> s/System\nAdmin/DevOps\nEngineer/
> s/System Admin/DevOps Engineer/
> ' data4.txt
On Tuesday, the Linux DevOps
Engineer group meeting will be held.
All DevOps Engineers should attend.
$
```

This example shows the exclamation mark used with the `N` command, along with the dollar sign (`$`) special address. The dollar sign represents the last line of text in the data stream, so when the `sed` editor reaches the last line, it doesn't execute the `N` command. However, for all other lines, it does execute the command.

Using this technique, you can reverse the order of text lines in a data stream. To reverse the order of the lines as they appear in the text stream (display the last line first and the first line last), you need to do some fancy footwork using the hold space.

To accomplish this, use `sed` to

1. Place a text line in the pattern space.
2. Copy the line in the pattern space into the hold space.
3. Put the next line of text in the pattern space.
4. Append the hold space to the pattern space.
5. Copy everything in the pattern space into the hold space.
6. Repeat steps 3 through 5 until you've put all the lines in reverse order in the hold space.
7. Retrieve the lines, and print them.

Figure 21-1 diagrams what this looks like in more detail.

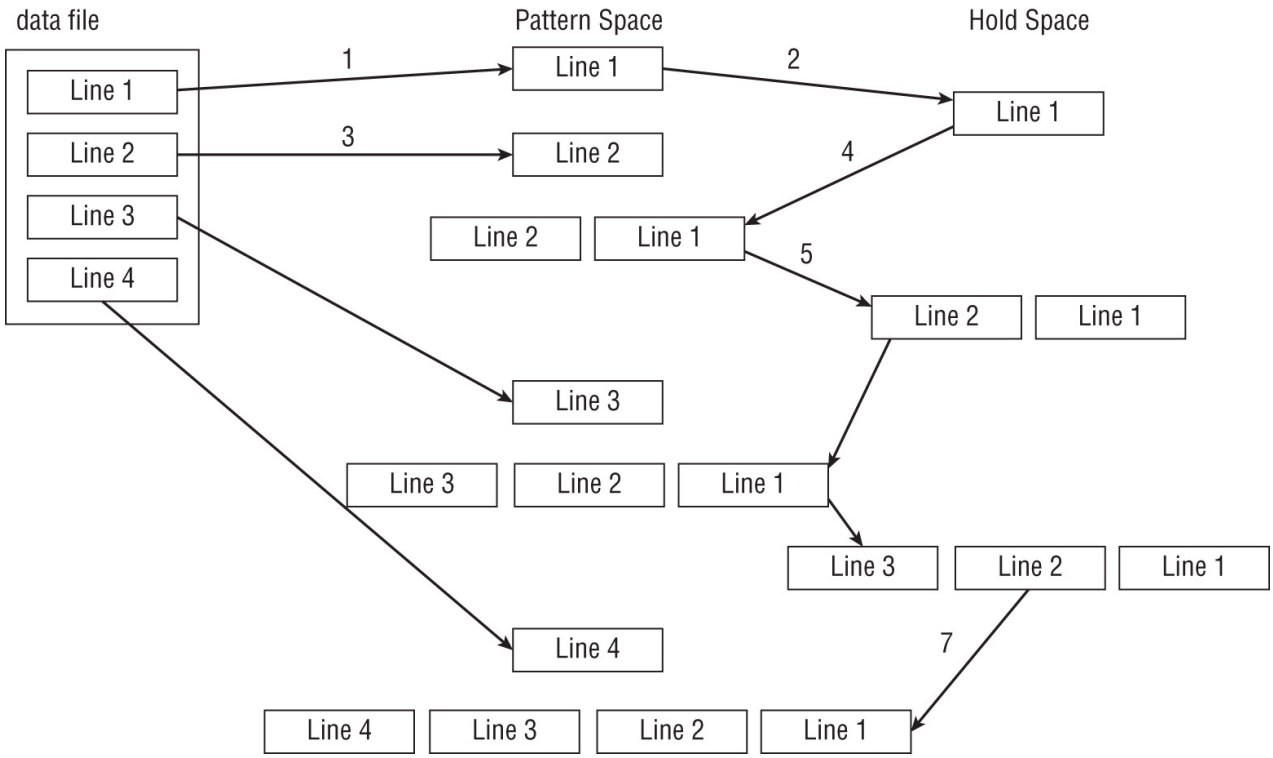

FIGURE 21-1 Reversing the order of a text file using the hold space

When using this technique, you do not want to print lines as they are processed. This means using the `-n` command-line option for `sed` . The next thing to determine is how to append the hold space text to the pattern space text. This is done by using the `G` command. The only problem is that you don't want to append the hold space to the first line of text processed. This is easily solved by using the exclamation mark command:

```
1!G
```

The next step is to copy the new pattern space (the text line with the appended reverse lines) into the hold space. This is simple enough; just use the `h` command.

When you've got the entire data stream in the pattern space in reverse order, you just need to print the results. You know you have the entire data stream in the pattern space when you've reached the last line in the data stream. To print the results, just use the following command:

```
$p
```

Those are the pieces we need to create our line-reversing `sed` editor script. Now we'll try it out in a test run:

```
$ cat data2.txt
Header Line
First Data Line
Second Data Line
End of Data Lines
$
$ sed -n '{1!G ; h ; $p }' data2.txt
End of Data Lines
Second Data Line
First Data Line
Header Line
$
```

The `sed` editor script performed as expected. The output from the script reverses the original lines in the text file. This demonstrates the power of using the hold space in your `sed` scripts. It provides an easy way to manipulate the order of lines in the script output.

> **NOTE**
>
> **In case you're wondering, a Bash shell command can perform the function of reversing a text file. The `tac` command displays a text file in reverse order. You probably noticed the clever name of the command because it performs the reverse function of the `cat` command.**

Changing the Flow

Normally, the `sed` editor processes commands starting at the top and proceeding toward the end of the script (the exception is the `D` command, which forces the `sed` editor to return to the top of the script without reading a new line

of text). The `sed` editor provides a method for altering the flow of the command script, producing a result similar to that of a structured programming environment.

Branching

In the previous section, you saw how the exclamation mark command is used to negate the effect of a command on a line of text. The `sed` editor provides a way to negate an entire section of commands, based on an address, an address pattern, or an address range. This allows you to perform a group of commands only on a specific subset within the data stream.

Here's the format of the branch (`b`) command:

```
[address]b [label]
```

The *`address`* parameter determines which line or lines of data trigger the branch (`b`) command. The *`label`* parameter defines the location within the script to which to branch. If the *`label`* parameter is not present, the branch (`b`) command skips the line or lines of data that triggered the branch, and goes on to process the other text lines.

Here is an example using the *`address`* parameter with the branch command but no *`label`*:

```
$ cat data2.txt
Header Line
First Data Line
Second Data Line
End of Data Lines
$
$ sed '{2,3b ;
> s/Line/Replacement/}
> ' data2.txt
Header Replacement
First Data Line
Second Data Line
End of Data Replacements
$
```

The branch (`b`) command skips the substitution commands for the second and third lines in the data stream.

Instead of going to the end of the script, you can define a *`label`* providing a location for the branch command to jump. Labels start with a colon and can be up to seven characters in length:

```
:label2
```

To specify the *`label`*, just add it after the `b` command. Using labels allows you to provide alternative commands to process data that match the branch *`address`* but still process other text lines using the original commands in the script:

```
$ sed '{/First/b jump1 ;
> s/Line/Replacement/
> :jump1
> s/Line/Jump Replacement/}
> ' data2.txt
Header Replacement
First Data Jump Replacement
Second Data Replacement
End of Data Replacements
$
```

The branch (`b`) command specifies that the program should jump to the script line labeled `jump1` if the matching text `First` appears in the line. If the branch command *`address`* doesn't match, the `sed` editor continues processing commands in the script, including the command *after* the branch label, `jump1` . (Thus, both substitution commands are processed on lines that don't match the branch *`address`*.)

If a line matches the branch *`address`*, the `sed` editor branches to the labeled line, `jump1` . Thus, only the last substitution command is executed for lines matching the branch *`address`*.

The example shows branching to a label further down in the `sed` script. You can also branch to a label that appears earlier in the script, thus creating a looping effect:

```
$ echo "This, is, a, test, to, remove, commas." |
> sed -n {'
> :start
> s/,//1p
> b start
> }'
This is, a, test, to, remove, commas.
This is a, test, to, remove, commas.
This is a test, to, remove, commas.
This is a test to, remove, commas.
This is a test to remove, commas.
This is a test to remove commas.
^C
$
```

Each script iteration removes the first occurrence of a comma from the text string and prints the string. There's one catch to this script — it never ends. This situation creates an endless loop, searching for commas until you manually

stop it by sending a signal with the Ctrl+C key combination.

To prevent this problem, specify an address pattern for the branch (`b`) command. If the pattern isn't present, the branching stops:

```
$ echo "This, is, a, test, to, remove, commas." |
> sed -n {'
> :start
> s/,//1p
> /,/b start
> }'
This is, a, test, to, remove, commas.
This is a, test, to, remove, commas.
This is a test, to, remove, commas.
This is a test to, remove, commas.
This is a test to remove, commas.
This is a test to remove commas.
$
```

Now the branch command branches only if there's a comma in the line. After the last comma has been removed, the branch (`b`) command doesn't execute, allowing the script to properly finish.

Testing

Similar to the branch command, the test (`t`) command is also used to modify the flow of the `sed` editor script. Instead of jumping to a *`label`* based on an *`address`*, the test (`t`) command jumps to a *`label`* based on the outcome of a preceding substitution command.

If the substitution command successfully matches and substitutes a pattern, the test command branches to the specified label. If the substitution command doesn't match the specified pattern, the test command doesn't branch.

The test (`t`) command uses the same format as the branch command:

```
[address]t [label]
```

As with the branch command, if you don't specify a *`label`*, `sed` jumps to the end of the script's commands, but only if the test succeeds.

The test (`t`) command provides a cheap way to perform a basic `if-then` statement on the text in the data stream. For example, if you don't need to make a substitution if another substitution was made, the test command, without a specified `label` , can help:

```
$ sed '{s/First/Matched/ ; t
> s/Line/Replacement/}
> ' data2.txt
Header Replacement
Matched Data Line
Second Data Replacement
End of Data Replacements
$
```

The first substitution command looks for the pattern text `First` . If it matches the pattern in the line, it replaces the text, and the test (`t`) command jumps over the second substitution command. If the first substitution command doesn't match the text pattern, the second substitution command is processed.

Using the test command, we can clean up the loop we tried using the branch command:

```
$ echo "This, is, a, test, to, remove, commas." |
> sed -n '{
> :start
> s/,//1p
> t start
> }'
This is, a, test, to, remove, commas.
This is a, test, to, remove, commas.
This is a test, to, remove, commas.
This is a test to, remove, commas.
This is a test to remove, commas.
This is a test to remove commas.
$
```

When there are no more comma substitutions to make, the test command doesn't branch, and the processing ends.

Replacing via a Pattern

We've covered how to use patterns in the `sed` commands to replace text in the data stream. However, when using wildcard characters it's not easy to know exactly what text will match the pattern.

For example, say that you want to place double quotation marks around a word you match in a line. That's simple enough if you're just looking for one word in the pattern to match:

```
$ echo "The cat sleeps in his hat." |
> sed 's/cat/"cat"/'
The "cat" sleeps in his hat.
$
```

But what if you use a wildcard character (.) in the pattern to match more than one word?

```
$ echo "The cat sleeps in his hat." |
> sed 's/.at/".at"/g'
The ".at" sleeps in his ".at".
$
```

The substitution string used the dot wildcard character to match any occurrence of a letter followed by “at”. Unfortunately, the replacement string doesn't match the wildcard character value of the matching word.

Using the ampersand

The `sed` editor has a solution for you. The ampersand symbol (`&`) is used to represent the matching pattern in the substitution command. Whatever text matches the pattern defined, you can use the ampersand symbol to recall it in the replacement pattern. This lets you manipulate whatever word matches the pattern defined:

```
$ echo "The cat sleeps in his hat." |
> sed 's/.at/"&"/g'
The "cat" sleeps in his "hat".
$
```

When the pattern matches the word `cat` , `"cat"` appears in the substituted word. When it matches the word `hat` , `"hat"` appears in the substituted word.

Replacing individual words

The ampersand symbol retrieves the entire string that matches the pattern you specify in the substitution command. Sometimes, you'll want to retrieve only a subset of the string. You can do that, too, but it's a little tricky.

The `sed` editor uses parentheses to define a substring component within the substitution pattern. You can then reference each substring component using a special character in the replacement pattern. The replacement character consists of a backslash and a number. The number indicates the substring component's position. The `sed` editor assigns the first component the character `\1` , the second component the character `\2` , and so on.

> **NOTE**
>
> **When you use parentheses in the substitution command, you must use the escape character to identify them as grouping characters and not normal parentheses. This is the reverse of when you escape other special characters.**

Look at an example of using this feature in a `sed` editor script:

```
$ echo "The Guide to Programming" |
> sed '
> s/\(Guide to\) Programming/\1 DevOps/'
The Guide to DevOps
$
```

This substitution command uses one set of parentheses around `Guide To` identifying it as a substring component. It then uses the `\1` in the replacement pattern to recall the first identified component. This isn't too exciting, but it can really be useful when working with wildcard patterns.

Suppose you need to replace a phrase with just a single word that's a substring of the phrase, but that substring just happens to be using a wildcard character. In such cases, using substring components is a lifesaver:

```
$ echo "That furry cat is pretty." |
> sed 's/furry \(.at\)/\1/'
That cat is pretty.
$
$ echo "That furry hat is pretty." |
> sed 's/furry \(.at\)/\1/'
That hat is pretty.
$
```

In this situation, you can't use the ampersand symbol, because it would replace the entire matching pattern. The substring component provides the answer, allowing you to select just which part of the pattern to use as the replacement pattern.

This feature can be especially helpful when you need to insert text between two or more substring components. Here's a script that uses substring components to insert a comma in long numbers:

```
$ echo "1234567" | sed '{
> :start
> s/\(.*[0-9]\)\([0-9]\{3\}\)/\1,\2/
> t start}'
1,234,567
$
```

The script divides the matching pattern into two components:

```
.*[0-9]
```

```
[0-9]{3}
```

This pattern looks for two substrings. The first substring is any number of characters ending in a digit. The second substring is a series of three digits (see Chapter 20, “Regular Expressions,” for information about how to use braces in a regular expression). If this pattern is found in the text, the replacement text puts a comma between the two components, each identified by its component position. The script uses the test (`t`) command to iterate through the number until all commas have been put in their proper place.

Placing *sed* Commands in Scripts

Now that you've seen the various parts of the `sed` editor, it's time to put them together and use them in your shell scripts. This section demonstrates some of the features that you should know about when using the `sed` editor in your Bash shell scripts.

Using wrappers

You may have noticed that trying to implement a `sed` editor script can be cumbersome, especially if the script is long. Instead of having to retype the entire script each time you want to use it, you can place the `sed` editor command in a shell script *wrapper*. The wrapper acts as a go-between for the `sed` editor script and the command line. A shell script wrapper, `ChangeScriptShell.sh` , was used back in Chapter 19 as a practical example.

Once inside the shell script, you can use normal shell variables and parameters with your `sed` editor scripts. Here's an example of using the command-line parameter variable as the input to a `sed` script:

```
$ cat reverse.sh
#!/bin/bash
# Shell wrapper for sed editor script
# to reverse test file lines.
#
sed -n '{1!G; h; $p}' $1
#
exit
$
```

The shell script called `reverse.sh` uses the `sed` editor script to reverse text lines in a data stream. It uses the `$1` shell parameter to retrieve the first parameter from the command line, which should be the name of the file to reverse:

```
$ cat data2.txt
Header Line
First Data Line
Second Data Line
End of Data Lines
$
$ ./reverse.sh data2.txt
End of Data Lines
Second Data Line
First Data Line
Header Line
$
```

Now you can easily use the `sed` editor script on any file without having to constantly retype the entire `sed` command line.

Redirecting *sed* output

By default, the `sed` editor outputs the results of the script to `STDOUT` . You can employ all the standard methods of redirecting the output of the `sed` editor in your shell scripts.

You can use dollar sign/parenthesis, `$()` , to redirect the output of your `sed` editor command to a variable for use later in the script. The following is an example of using the `sed` script to add commas to the result of a numeric computation:

```
$ cat fact.sh
#!/bin/bash
# Shell wrapper for sed editor script
# to calculate a factorial, and
# format the result with commas.
#
factorial=1
counter=1
number=$1
#
while [ $counter -le $number ]
do
   factorial=$[ $factorial * $counter ]
   counter=$[ $counter + 1 ]
done
#
result=$(echo $factorial |
sed '{
:start
s/\(.*[0-9]\)\([0-9]\{3\}\)/\1,\2/
t start
```

```
}')
#
echo "The result is $result"
#
exit
$
$ ./fact.sh 20
The result is 2,432,902,008,176,640,000
$
```

After you use the normal factorial calculation script, the result of that script is used as the input to the `sed` editor script, which adds commas. This value is then used in the `echo` statement to produce the result. And how nice to now have that long `sed` script within a Bash shell script so that you can use it without typing all the `sed` commands in again!

Creating *sed* Utilities

As you've seen in the short examples presented so far in this chapter, you can do lots of cool data-formatting things with the `sed` editor. This section shows a few handy well-known `sed` editor scripts for performing common data-handling functions.

Spacing with double lines

To start things off, look at a simple `sed` script to insert a blank line between lines in a text file:

```
$ sed 'G' data2.txt
Header Line

First Data Line

Second Data Line

End of Data Lines

$
```

That was pretty simple! The key to this trick is the default value of the hold space. Recall that the `G` command simply appends the contents of the hold space to the current pattern space contents. When you start the `sed` editor, the hold space contains an empty line. By appending that to an existing line, you create a blank line after the existing line.

You may have noticed that this script also adds a blank line to the last line in the data stream, producing a blank line at the end of the file. If you want to get rid of this, you can use the negate symbol and the last line symbol to ensure that the script doesn't add the blank line to the last line of the data stream:

```
$ sed '$!G' data2.txt
Header Line

First Data Line

Second Data Line

End of Data Lines
$
```

Now that looks a little better. As long as the line isn't the last line, the `G` command appends the contents of the hold space. When the `sed` editor gets to the last line, it skips the `G` command.

Spacing files that may have blanks

To take double spacing one step further, what if the text file already has a few blank lines but you want to double-space all the lines? If you use the previous script, you'll get some areas that have too many blank lines, because each existing blank line gets doubled:

```
$ cat data6.txt
Line one.
Line two.

Line three.
Line four.
$
$ sed '$!G' data6.txt
Line one.

Line two.



Line three.

Line four.
$
```

Now we have three blank lines where the original blank line was located. The solution to this problem is to first

delete any blank lines from the data stream and then use the `G` command to insert new blank lines after all the lines. To delete existing blank lines, we just need to use the `d` command with a pattern that matches a blank line:

```
/^$/d
```

This pattern uses the start line tag (the caret) and the end line tag (the dollar sign). Adding this pattern to the script produces the desired results:

```
$ sed '/^$/d ; $!G' data6.txt
Line one.

Line two.

Line three.

Line four.
$
```

Perfect! It works just as desired.

Numbering lines in a file

Chapter 19 showed you how to use the equal sign to display the line numbers of lines in the data stream:

```
$ sed '=' data2.txt
1
Header Line
2
First Data Line
3
Second Data Line
4
End of Data Lines
$
```

This can be a little awkward to read, because the line number is on a line above the actual line in the data stream. A better solution is to place the line number on the same line as the text.

Now that you've seen how to combine lines using the `N` command, it shouldn't be too hard to utilize that information in the `sed` editor script. The trick to this utility, however, is that you can't combine the two commands in the same script.

After you have the output for the equal sign command, you can pipe the output to another `sed` editor script that uses the `N` command to combine the two lines. You also need to use the substitution (`s`) command to replace the newline character with either a space or a tab character. Here's what the final solution looks like:

```
$ sed '=' data2.txt | sed 'N; s/\n/ /'
1 Header Line
2 First Data Line
3 Second Data Line
4 End of Data Lines
$
```

Now that looks much better. This is a great little utility to have around when you're working on programs where you need to see the line numbers used in error messages.

There are Bash shell commands that can also add line numbers. However, they add some additional (and potentially unwanted) spacing:

```
$ nl data2.txt
     1  Header Line
     2  First Data Line
     3  Second Data Line
     4  End of Data Lines
$
$ cat -n data2.txt
     1  Header Line
     2  First Data Line
     3  Second Data Line
     4  End of Data Lines
$
$ nl data2.txt | sed 's/     //; s/\t/ /'
1 Header Line
2 First Data Line
3 Second Data Line
4 End of Data Lines
$
```

The `sed` editor script handles the output without any additional spacing. But if you want to use these utilities, `sed` is there to help you remove any unwanted spacing!

Printing last lines

So far, we've covered how to use the `p` command to print all the lines in a data stream or just lines that match a specific pattern. What if you just need to work with the last few lines of a long listing, such as a log file?

The dollar sign represents the last line of a data stream, so it's easy to display just the last line:

```
$ sed -n '$p' data2.txt
End of Data Lines
$
```

Now, how can you use the dollar sign symbol to display a set number of lines at the end of the data stream? The answer is to create a *rolling window*.

A rolling window is a common way to examine blocks of text lines in the pattern space by combining them using the `N` command. The `N` command appends the next line of text to the text already in the pattern space. After you have a block of 10 text lines in the pattern space, you can check to see if you're at the end of the data stream using the dollar sign. If you're not at the end, continue adding more lines to the pattern space while removing the original lines (remember the `D` command, which deletes the first line in the pattern space).

By looping through the `N` and `D` commands, you add new lines to the block of lines in the pattern space while removing old lines. The branch (`b`) command is the perfect fit for the loop. To end the loop, just identify the last line and use the `q` command to quit.

Here's what the final `sed` editor script looks like:

```
$ cat data7.txt
Line1
Line2
Line3
Line4
Line5
Line6
Line7
Line1
Line2
Line3
Line4
Line5
Line6
Line7
Line8
Line9
Line10
Line11
Line12
Line13
Line14
Line15
$
$ sed '{
> :start
> $q ; N ; 11,$D
> b start
> }' data7.txt
Line6
Line7
Line8
Line9
Line10
Line11
Line12
Line13
Line14
Line15
$
```

The script first checks whether the line is the last line in the data stream. If it is, the quit (`q`) command stops the loop. The `N` command appends the next line to the current line in the pattern space. The `11,$D` command deletes the first line in the pattern space if the current line is after line 10. This creates the sliding window effect in the pattern space. Thus, the `sed` program script displays only the last 10 lines of the `data7.txt` file.

Deleting lines

Another useful utility for the `sed` editor is to remove unwanted blank lines in a data stream. It's easy to remove all the blank lines from a data stream, but it takes a little ingenuity to selectively remove blank lines. This section shows you three quick `sed` editor scripts that you can use to help remove unwanted blank lines from your data.

Deleting consecutive blank lines

It can be a nuisance when extra blank lines crop up in data files. Often you have a data file that contains blank lines, but sometimes a data line is missing and produces too many blank lines (as you saw in the double-spacing example earlier).

The easiest way to remove consecutive blank lines is to check the data stream using a range address. In Chapter 19 we covered how to use ranges in addresses, including how to incorporate patterns in the address range. The `sed` editor executes the command for all lines that match within the specified address range.

The key to removing consecutive blank lines is to create an address range that includes a non-blank line and a blank

line. If the sed editor comes across this range, it shouldn't delete the line. However, for lines that don't match that range (two or more blank lines in a row), it should delete the lines.

Here's the script to do this:

```
/./,/^$/!d
```

The range is /./ to / ^ $/ . The start address in the range matches any line that contains at least one character. The end address in the range matches a blank line. Lines within this range aren't deleted.

Here's the script in action:

```
$ cat data8.txt
Line one.


Line two.

Line three.



Line four.
$
$ sed '/./,/^$/!d' data8.txt
Line one.

Line two.

Line three.

Line four.
$
```

No matter how many blank lines appear between lines of data in the file, the output places only one blank line between the lines.

Deleting leading blank lines

It is also a nuisance when data files contain multiple blank lines at the start of the file. Often when you are trying to import data from a text file into a database, the blank lines create null entries, throwing off any calculations using the data.

Removing blank lines from the top of a data stream is not a difficult task. Here's the script that accomplishes that function:

```
/./,$!d
```

The script uses an address range to determine what lines are deleted. The range starts with a line that contains a character and continues to the end of the data stream. Any line within this range is not deleted from the output. This means that any lines before the first line that contain a character are deleted.

Look at this simple script in action:

```
$ cat data9.txt


Line one.

Line two.
$
$ sed '/./,$!d' data9.txt
Line one.

Line two.
$
```

The test file contains two blank lines before the data lines. The script successfully removes both of the leading blank lines while keeping the blank line within the data intact.

Deleting trailing blank lines

Unfortunately, deleting trailing blank lines is not as simple as deleting leading blank lines. Just like printing the end of a data stream, deleting blank lines at the end of a data stream requires a little ingenuity and looping.

Before we start the discussion, let's see what the script looks like:

```
sed '{
:start
/^\n*$/{$d; N; b start }
}'
```

This may look a little odd to you at first. Notice that there are braces within the normal script braces. This allows you to group commands together within the overall command script. The group of commands applies to the specified address pattern. The address pattern matches any line that contains only a newline character. When one is found, if

it's the last line, the `d` command deletes it. If it's not the last line, the `N` command appends the next line to it, and the branch (`b`) command loops to the beginning to start over.

Here's the script in action:

```
$ cat data10.txt
Line one.
Line two.



$
$ sed '{
> :start
> /^\n*$/{$d; N; b start}
> }' data10.txt
Line one.
Line two.
$
```

The script successfully removed the blank lines from the end of the text file.

Removing HTML tags

These days, it's not uncommon to download text from a website to save or use as data in an application. Sometimes, however, when you download text from the website, you also get the HTML tags used to format the data. This can be a problem when all you want to see is the data.

A standard HTML web page contains several different types of HTML tags, identifying formatting features required to properly display the page information. Here's a sample of what an HTML file looks like:

```
$ cat data11.txt
<html>
<head>
<title>This is the page title</title>
</head>
<body>
<p>
This is the first line in the Web page.
This should provide some useful
information to use in our sed script.
</body>
</html>
$
```

HTML tags are identified by the less-than and greater-than symbols. Most HTML tags come in pairs. One tag starts the formatting process (for example, `<b>` for bolding), and another tag stops the formatting process (for example, `</b>` to turn off bolding).

Removing HTML tags creates a problem, however, if you're not careful. At first glance, you'd think that the way to remove HTML tags would be to just look for a text string that starts with a less-than symbol (<), ends with a greater-than symbol (>), and has data in between the symbols:

```
s/<.*>//g
```

Unfortunately, this command has some unintended consequences:

```
$ sed 's/<.*>//g' data11.txt





This is the  line in the Web page.
This should provide some
information to use in our sed script.


$
```

Notice that the title text is missing, along with the text that was bolded and italicized. The `sed` editor literally interpreted the script to mean any text between the less-than and greater-than sign, including other less-than and greater-than signs! Each time the text was enclosed in HTML tags (such as `<b>first</b>`), the `sed` script removed the entire text.

The solution to this problem is to have the `sed` editor ignore any embedded greater-than signs between the original tags. To do that, you can create a character class that negates the greater-than sign. This changes the script to

```
s/<[^>]*>//g
```

This script now works properly, displaying the data you need to see from the web page HTML code:

```
$ sed 's/<[^>]*>//g' data11.txt
```

```
This is the page title



This is the first line in the Web page.
This should provide some useful
information to use in our sed script.


$
```

That's a little better. To clean things up some, we can add a delete command to get rid of those pesky blank lines:

```
$ sed 's/<[^>]*>//g ; /^$/d' data11.txt
This is the page title
This is the first line in the Web page.
This should provide some useful
information to use in our sed script.
$
```

Now that's much more compact; there's only the data you need to see.

Working Through a Practical Example

For our practical example in this chapter, we'll use `sed` to scan Bash shell scripts. The purpose of this scan is to find commands that may be better off located within a function.

Back in Chapter 17, “Creating Functions,” we covered how to set up functions that can be called multiple times within a script. It's considered good form to put any duplicated code blocks within a function. That way, if a change is needed for the commands in the code block, you need to make those changes in only one place. Not only is this approach a time-saver, but it also reduces the chances of introducing errors that may occur if you change one block of code but not the others strewn about the script.

To keep things simple, we'll just look for three repeating lines of code within a file. Consider the following text file:

```
$ cat ScriptData.txt
Line 1
Line 2
Line 3
Line 4
Line 5
Line 6
Line 3
Line 4
Line 5
Line 7
Line 8
Line 3
Line 4
Line 5
Line 9
Line 10
Line 11
Line 12
$
```

If you look carefully, you'll notice that `Line 3` , `Line 4` , and `Line 5` are repeated three times throughout this file. To find those repeated lines, we'll merge each text line together with the next two following lines:

1. Read in the next line, but only if processing the text file's first line, using the multiline `N` command.
2. Read in the next line (this is the second read for the text file's first line) using the multiline `N` command. Now we have three lines within the pattern space.
3. Print the pattern space to `STDOUT` .
4. Delete the first line in the pattern space using the `D` command, which deletes the text within the pattern space up to and including the first newline, and then starts processing the `sed` script commands from the beginning.

The following command shows this process:

```
$ sed -n '{
> 1N
> N
> p
> D}
> ' ScriptData.txt
Line 1
Line 2
Line 3
Line 2
Line 3
Line 4
Line 3
Line 4
```

```
Line 5
Line 4
Line 5
Line 6
Line 5
Line 6
Line 3
Line 6
Line 3
Line 4
Line 3
Line 4
Line 5
Line 4
Line 5
[...]
Line 9
Line 10
Line 9
Line 10
Line 11
Line 10
Line 11
Line 12
$
```

The problem with this method is that it is hard for a human to differentiate what lines have been merged together. Because the lines still contain the newline character (`\n`), the output is nearly impossible to interpret.

To fix this issue, we'll replace the newline character at the end of each text line with a bell sound character (`\a`) using the substitute (`s`) command:

```
$ sed -n '{
> 1N
> N
> s/\n/\a/g
> p
> s/\a/\n/
> D}
> ' ScriptData.txt
Line 1Line 2Line 3
Line 2Line 3Line 4
Line 3Line 4Line 5
Line 4Line 5Line 6
Line 5Line 6Line 3
Line 6Line 3Line 4
Line 3Line 4Line 5
Line 4Line 5Line 7
Line 5Line 7Line 8
Line 7Line 8Line 3
Line 8Line 3Line 4
Line 3Line 4Line 5
Line 4Line 5Line 9
Line 5Line 9Line 10
Line 9Line 10Line 11
Line 10Line 11Line 12
$
```

Now, we're getting somewhere! Notice that although we replaced the newline characters with a bell sound character (globally) in the third line of the `sed` script, after printing the pattern space we had to switch the first bell sound character back to a newline. This is due to the delete (`D`) command, which needs the newline character after the first text line so that it won't delete the entire pattern space.

While it may be tempting to forge ahead to comparing the lines and finding duplicates, there is one more consideration. Shell script lines often have spaces or tabs within each line, as shown in this test text file:

```
$ cat ScriptDataB.txt
Line 1
Line 2
Line 3
Line 4
 Line 5
Line 6
     Line 3
     Line 4
  Line 5
Line 7
Line 8
        Line 3
        Line 4
        Line 5
Line 9
Line 10
Line 11
Line 12
$
```

These spaces and tabs will disrupt the matching process. But it's fairly easy to handle this situation. We'll just eliminate the spaces and tabs via a global (`g`) substitute (`s`) command:

```
$ sed -n '{
> 1N
> N
> s/ //g
> s/\t//g
> s/\n/\a/g
> p
> s/\a/\n/
> D}
> ' ScriptDataB.txt
Line1Line2Line3
Line2Line3Line4
Line3Line4Line5
Line4Line5Line6
Line5Line6Line3
Line6Line3Line4
Line3Line4Line5
Line4Line5Line7
Line5Line7Line8
Line7Line8Line3
Line8Line3Line4
Line3Line4Line5
Line4Line5Line9
Line5Line9Line10
Line9Line10Line11
Line10Line11Line12
$
```

Now we'll employ two other Bash shell commands to help us sort the file (`sort`) and find any duplicates (`uniq -d`):

```
$ sed -n '{
> 1N;
> N;
> s/ //g;
> s/\t//g;
> s/\n/\a/g;
> p
> s/\a/\n/;
> D}
> ' ScriptDataB.txt |
> sort | uniq -d |
> sed 's/\a/\n/g'
Line3
Line4
Line5
$
```

Perfect! The commands scanned the file and found three repeating lines. Now we can pretty this up, and store it in a shell wrapper. Here's one way to do it:

```
$ cat NeededFunctionCheck.sh
#!/bin/bash
# Checks for 3 duplicate lines in scripts.
# Suggest these lines be possibly replaced
# by a function.
#
tempfile=$2
#
#
sed -n '{
1N; N;
s/ //g; s/\t//g;
s/\n/\a/g; p;
s/\a/\n/; D}' $1>> $tempfile
#
sort $tempfile | uniq -d | sed 's/\a/\n/g'
#
rm -i $tempfile
#
exit
$
```

Notice that this script takes two parameters — the first (`$1`) for which file to scan, and the second (`$2`) to designate a temporary file to store the merged file lines. The reason we redirected `STDOUT` to a file, instead of directly into the `sort` command, is that it gives you the ability to keep the temporary file to see if your `sed` merging process (and any tweaks you make to it) are working correctly.

Before you start diving into modifying this script to try out variations, let's scan our test text file with it:

```
$ ./NeededFunctionCheck.sh ScriptDataB.txt TempFile.txt
Line3
Line4
Line5
rm: remove regular file 'TempFile.txt'? Y
```

```
$
```

This is exactly what we are wanting. Now, let's try scanning a real script and see how it does:

```
$ ./NeededFunctionCheck.sh CheckMe.sh TempFile.txt
echo"Usage:./CheckMe.shparameter1parameter2"
echo"Exitingscript..."
exit
rm: remove regular file 'TempFile.txt'? Y
$
```

This is a little harder to read, because we eliminated *all* the spaces within each script line. (One script improvement you can make is to only remove multiple spaces between characters.) However, it gives us enough of a shove in the right direction. We know that these three lines are repeated within the script. And the script needs to be reviewed to possibly replace the repeated commands with a function.

You can make lots of potential improvements within this script. Once you've got your modifications in place and working, remove that temporary file to make it faster. How about producing line numbers to show where the code is located within the script? You could even make the information into a fancy report...but we recommend you wait until you read through the next chapter, “Advanced `gawk` ,” before you try that script improvement.

Summary

The `sed` editor provides some advanced features that allow you to work with text patterns across multiple lines. This chapter showed you how to use the multiline next (`N`) command to retrieve the next line in a data stream and place it in the pattern space. Once it's in the pattern space, you can perform complex substitution (`s`) commands to replace phrases that span more than one line of text.

The multiline delete (`D`) command allows you to remove the first line when the pattern space contains two or more lines. This is a convenient way to iterate through multiple lines in the data stream. Similarly, the multiline print (`P`) command allows you to print just the first line when the pattern space contains two or more lines of text. The combination of the multiline commands allows you to iterate through the data stream and create a multiline substitution system.

Next, we covered the hold space. The hold space allows you to set aside a line of text while processing more lines of text. You can recall the contents of the hold space at any time and either replace the text in the pattern space or append the contents of the hold space to the text in the pattern space. Using the hold space allows you to sort through data streams, reversing the order of text lines as they appear in the data.

Next we reviewed the various `sed` editor flow control commands. The branch (`b`) command provides a way for you to alter the normal flow of `sed` editor commands in the script, creating loops or skipping commands under certain conditions. The test (`t`) command provides an `if-then` type of statement for your `sed` editor command scripts. The test (`t`) command branches only if a prior substitution (`s`) command succeeds in replacing text in a line.

The chapter continued with a discussion of how to use `sed` scripts in your shell scripts. A common technique for large `sed` scripts is to place the script in a shell wrapper. You can use command-line parameter variables within the `sed` script to pass shell command-line values. This creates an easy way to utilize your `sed` editor scripts directly from the command line, or even from other shell scripts.

We concluded the chapter with a look at creating common `sed` utilities, which allow you to do lots of processing of text files. Some features include numbering lines in a file in a more human-readable format, printing last text file lines, and removing HTML tags.

The next chapter digs deeper into the `gawk` world. The `gawk` program supports many features of higher-level programming languages. You can create some pretty involved data manipulation and reporting programs just by using `gawk` . The chapter describes the various programming features and demonstrates how to use them to generate your own fancy reports from simple data.